JWT Authentication & Authorization

Question 1 — Implement JWT Authentication in ASP.NET Core Web API

Scenario: Build a microservice with secure login using JWT-based authentication.

Step 1 — Create the Project and Install Package

```
dotnet new webapi -n JwtAuthDemo
cd JwtAuthDemo
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

```
Build succeeded.
Package 'Microsoft.AspNetCore.Authentication.JwtBearer' added.
Project 'JwtAuthDemo' created at JwtAuthDemo/
```

Step 2 — appsettings.json

```
{
  "Jwt": {
    "Key": "ThisIsASecretKeyForJwtToken",
    "Issuer": "MyAuthServer",
    "Audience": "MyApiUsers",
"DurationInMinutes": 60
  },
  "Logging": {     "LogLevel": { "Default":
"Information" }
  }
}
```

Step 3 — Models/LoginModel.cs and Models/User.cs

```
// Models/LoginModel.cs
public class LoginModel
{
    public string Username { get; set; } =
string.Empty;     public string Password { get; set; }
= string.Empty; }

// Models/User.cs  (in-memory user store for
demo) public static class UserStore {
    public static bool IsValid(string username, string password)
=> username == "admin" && password == "password123";
}
```

Step 4 — Program.cs — Register JWT Authentication

```
using
Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens; using
System.Text;

var builder = WebApplication.CreateBuilder(args);

// JWT Authentication  builder.Services.AddAuthentication("Bearer")
.AddJwtBearer("Bearer", options =>
    {
        options.TokenValidationParameters = new
TokenValidationParameters         {
            ValidateIssuer           = true,
            ValidateAudience         = true,
```

```
            ValidateLifetime         = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer    = builder.Configuration["Jwt:Issuer"],
            ValidAudience  = builder.Configuration["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(
Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!))
        };
});

builder.Services.AddAuthorization();
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI();
app.UseAuthentication();   // must come before
UseAuthorization app.UseAuthorization();
app.MapControllers(); app.Run();
```

Step 5 — Controllers/AuthController.cs

```
using Microsoft.AspNetCore.Mvc; using
Microsoft.IdentityModel.Tokens; using
System.IdentityModel.Tokens.Jwt;
using System.Security.Claims; using
System.Text;

[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly IConfiguration _config;
    public AuthController(IConfiguration config) => _config = config;

    [HttpPost("login")]
    public IActionResult Login([FromBody] LoginModel
model)     {
        if (!UserStore.IsValid(model.Username, model.Password))
            return Unauthorized(new { message = "Invalid username or password." });

        var token =
GenerateJwtToken(model.Username);         return
Ok(new { Token = token });     }

    private string GenerateJwtToken(string
username)     {
        var claims = new[]
{
            new Claim(ClaimTypes.Name, username),
            new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
};

        var key   = new SymmetricSecurityKey(
Encoding.UTF8.GetBytes(_config["Jwt:Key"]!));         var creds = new
SigningCredentials(key, SecurityAlgorithms.HmacSha256);

        var token = new JwtSecurityToken(
issuer:             _config["Jwt:Issuer"],
audience:           _config["Jwt:Audience"],
claims:             claims,
            expires:            DateTime.UtcNow.AddMinutes(
```

```
double.Parse(_config["Jwt:DurationInMinutes"]!)),             signingCredentials:
creds);

        return new
JwtSecurityTokenHandler().WriteToken(token);     }
}
```

Output — Login Request and Response

```
# POST http://localhost:5000/api/auth/login
# Body (JSON):
{
  "username": "admin",
"password": "password123"
}
```

```
HTTP 200 OK
{   "token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
            eyJodHRwOi8vc2NoZW1hcy54bWxzb2FwLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2
NsYW1tcy9uYW1lIjoiYWRtaW4iLCJqdGkiOiJhYmMxMjMiLCJleHAiOjE3MDAwMDAw
MDB9.SIG"
}

# Wrong credentials:
HTTP 401 Unauthorized
{ "message": "Invalid username or password." }
```

.

Question 2 — Secure an API Endpoint Using JWT

Step 1 — Controllers/SecureController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class SecureController :
ControllerBase {
    [HttpGet("data")]     [Authorize]                    //
requires a valid Bearer token     public IActionResult
GetSecureData()     {
        var username =
User.Identity?.Name;         return
Ok(new         {
            message  = "This is protected
data.",             user     = username,
accessed = DateTime.UtcNow         });
    }
}
```

Step 2 — Test Without Token

```
# GET http://localhost:5000/api/secure/data
# Headers: (none)
HTTP 401 Unauthorized

Response Headers:
  WWW-Authenticate: Bearer error="invalid_token"

Response Body: (empty — no token provided)
```

Step 3 — Test With Valid Token

```
# GET http://localhost:5000/api/secure/data # Headers:   Authorization:
Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...<token from Q1>
HTTP 200 OK
{   "message": "This is protected
data.",
  "user":    "admin",   "accessed":
"2024-01-01T10:30:00Z"
}
```

Step 4 — Test With Tampered Token

```
# GET http://localhost:5000/api/secure/data # Headers:   Authorization:
Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.TAMPERED.SIG
HTTP 401 Unauthorized

Console log:   warn:
Microsoft.AspNetCore.Authentication.JwtBearer[3]
Exception occurred while processing message.
        System.Exception: IDX10511: Signature validation failed.

Response: access denied — token signature is invalid.
```

The [Authorize] attribute tells ASP.NET Core to run the authentication middleware before the action executes. If no valid Bearer token is present, the request is rejected with 401 before the controller code even runs.

Question 3 — Add Role-Based Authorization

Step 1 — Add Role Claim to Token Generation (AuthController.cs)

```
// Updated GenerateJwtToken — include role claim
private string GenerateJwtToken(string username, string role =
"User") {
    var claims = new[]
{
        new Claim(ClaimTypes.Name,               username),
        new Claim(ClaimTypes.Role,              role),      //  role claim added
new Claim(JwtRegisteredClaimNames.Jti,   Guid.NewGuid().ToString())     };

    var key   = new SymmetricSecurityKey(
Encoding.UTF8.GetBytes(_config["Jwt:Key"]!));     var creds = new
SigningCredentials(key, SecurityAlgorithms.HmacSha256);

    var token = new JwtSecurityToken(
issuer:             _config["Jwt:Issuer"],
audience:           _config["Jwt:Audience"],
claims:             claims,
        expires:            DateTime.UtcNow.AddMinutes(60),
signingCredentials: creds);

    return new
JwtSecurityTokenHandler().WriteToken(token); }
```

Step 2 — Updated Login to Return Role-Aware Token

```
[HttpPost("login")]
public IActionResult Login([FromBody] LoginModel
model) {
    if (!UserStore.IsValid(model.Username, model.Password))
return Unauthorized(new { message = "Invalid credentials." });

    // Assign role based on username (in real app: look up from
database)     var role  = model.Username == "admin" ? "Admin" : "User";
var token = GenerateJwtToken(model.Username, role);     return Ok(new {
Token = token, Role = role }); }
```

Step 3 — Controllers/AdminController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class AdminController : ControllerBase
{
    [HttpGet("dashboard")]     [Authorize(Roles = "Admin")]
// only Admin role can access     public IActionResult
GetAdminDashboard()     {         return Ok(new         {
            message = "Welcome to the admin dashboard.",
user    = User.Identity?.Name,
            role    = User.FindFirst(System.Security.Claims.ClaimTypes.Role)?.Value
});
    }

    [HttpGet("report")]
    [Authorize(Roles = "Admin,Manager")]  // multiple roles
allowed     public IActionResult GetReport()     {
        return Ok(new { message = "Report data — Admin or Manager only."
});     }
}
```

Output — Admin Login

```
# POST http://localhost:5000/api/auth/login
{
  "username": "admin",
"password": "password123"
}
HTTP 200 OK
{                                          "token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",   "role":
"Admin"
}

Decoded JWT payload:
{
  "http://schemas.xmlsoap.org/ws/2005/05/identity/claims/name": "admin",
"http://schemas.microsoft.com/ws/2008/06/identity/claims/role": "Admin",
  "jti": "abc-123",
"exp": 1700000000
}
```

Output — Access Admin Dashboard with Admin Token

```
# GET
http://localhost:5000/api/admin/dashboard #
Authorization: Bearer <admin-token>
HTTP 200 OK
{   "message": "Welcome to the admin
dashboard.",
  "user":    "admin",
"role":    "Admin"
}
```

Output — Access Admin Dashboard with User Token

```
# Login as a regular user first # POST /api/auth/login  {
"username": "john", "password": "pass" } # Then:
# GET http://localhost:5000/api/admin/dashboard #
Authorization: Bearer <user-token>   (role = "User")
HTTP 403 Forbidden

Note:
  401 Unauthorized = no token / invalid token (not authenticated)
  403 Forbidden    = valid token but wrong role (authenticated, not authorized)

The user is logged in but does not have the Admin role — access is denied.
```

Always use 403 for authorization failures (wrong role) and 401 for authentication failures (missing/bad token). ASP.NET Core handles this distinction automatically when [Authorize(Roles=...)] is used.

Question 4 — Validate JWT Token Expiry and Handle Unauthorized Access

Step 1 — Configure JwtBearerEvents in Program.cs

```
builder.Services.AddAuthentication("Bearer"
)     .AddJwtBearer("Bearer", options =>
{
        options.TokenValidationParameters = new
TokenValidationParameters         {
            ValidateIssuer           = true,
            ValidateAudience         = true,
            ValidateLifetime         = true,
            ValidateIssuerSigningKey = true,
```

```
            ValidIssuer              = builder.Configuration["Jwt:Issuer"],
ValidAudience            = builder.Configuration["Jwt:Audience"],
            IssuerSigningKey         = new SymmetricSecurityKey(
Encoding.UTF8.GetBytes(
                                                builder.Configuration["Jwt:Key"]!)),
// Allow zero clock skew for strict expiry — default is 5 minutes
ClockSkew = TimeSpan.Zero
        };

        // Custom JWT Bearer Events        options.Events = new JwtBearerEvents
{             // Fires when token validation fails (expired, bad signature, etc.)
OnAuthenticationFailed = context =>             {
                if (context.Exception is SecurityTokenExpiredException)
{                     // Add a custom response header to signal token
expiry                     context.Response.Headers.Append("Token-Expired",
"true");                 }
                return Task.CompletedTask;
},

            // Fires when the request has no token or the scheme doesn't match
OnChallenge = context =>
            {                   // Suppress the default 401 response
and write our own                          context.HandleResponse();
context.Response.StatusCode  = 401;
                context.Response.ContentType = "application/json";

                var message = context.AuthenticateFailure is
SecurityTokenExpiredException                     ? "Token has expired. Please log in
again."                     : "Access denied. A valid JWT token is required.";

                var body =
System.Text.Json.JsonSerializer.Serialize(                     new
{ error = message });                 return
context.Response.WriteAsync(body);             },

            // Fires when the user is authenticated but lacks the required
role/policy             OnForbidden = context =>             {
                context.Response.StatusCode  = 403;
                context.Response.ContentType = "application/json";
var body = System.Text.Json.JsonSerializer.Serialize(
                    new { error = "You do not have permission to access this resource."
});                 return context.Response.WriteAsync(body);             }
        };
    });
```

Step 2 — Test: No Token Provided

```
# GET http://localhost:5000/api/secure/data
# Headers: (no Authorization header)
```

```
HTTP 401 Unauthorized Content-
Type: application/json

{   "error": "Access denied. A valid JWT token is
required."
}
```

Step 3 — Test: Expired Token

To simulate expiry quickly, temporarily set DurationInMinutes to 0 in appsettings.json, login to get a token, then restore the value and test with that token.

```
# Temporary: "DurationInMinutes": 0  (token expires immediately)
# POST /api/auth/login → get token
# Restore: "DurationInMinutes": 60
# Then:
```

```
# GET http://localhost:5000/api/secure/data
# Authorization: Bearer <expired-token>

HTTP 401 Unauthorized
Content-Type: application/json Token-Expired: true
custom response header

{   "error": "Token has expired. Please log in
again."
}

Console log:
  Microsoft.AspNetCore.Authentication.JwtBearer[3]
Exception occurred while processing message.
  Microsoft.IdentityModel.Tokens.SecurityTokenExpiredException:
  IDX10223: Lifetime validation failed. The token is expired.
  ValidTo:  '01/01/2024 10:00:00'
  Current:  '01/01/2024 10:05:00'
```

Step 4 — Test: Invalid / Tampered Token

```
# GET http://localhost:5000/api/secure/data #
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.INVALID.BADSIG

HTTP 401 Unauthorized Content-
Type: application/json

{   "error": "Access denied. A valid JWT token is
required."
}

Console log:
  Microsoft.IdentityModel.Tokens.SecurityTokenSignatureKeyNotFoundException:
  IDX10501: Signature validation failed.
```

Step 5 — Test: Valid Token but Wrong Role (403)

```
# GET http://localhost:5000/api/admin/dashboard #
Authorization: Bearer <valid-user-token-with-role=User>

HTTP 403 Forbidden Content-
Type: application/json

{   "error": "You do not have permission to access this
resource."
}
```

Complete Flow Summary

```
Flow 1 — Successful Access:

  Client → POST /api/auth/login (username + password)
           200 OK { "token": "eyJ..." }

  Client → GET  /api/secure/data  (Authorization: Bearer eyJ...)
           200 OK { "message": "This is protected data.", "user": "admin" }

Flow 2 — Expired Token:

  Client → GET  /api/secure/data  (Authorization: Bearer <expired>)
           401 Unauthorized  (Token-Expired: true header)
             { "error": "Token has expired. Please log in again." }

  Client → POST /api/auth/login again → get fresh token → retry

Flow 3 — Wrong Role:

  Client → GET  /api/admin/dashboard  (Authorization: Bearer <user-token>)
           403 Forbidden             { "error": "You do not have permission
to access this resource." }
```